

FRONT PAGE

CERTIFICATE PAGE

ACKNOWLEDGEMENT

ABSTRACT

This study introduces a web tool designed to assess acne severity by analyzing face photos, combining image analysis with machine learning in a practical framework. Its main aim is to give people an easy, quick method to check their skin status at home - without needing instant medical advice. Through patterns found in pictures, the platform delivers early feedback, supporting better tracking and personal decisions about skincare routines. Central to its function is a deep learning model built specifically to detect and rate breakouts accurately.

The model divides skin issues into three types - Clear, Moderate, or Severe depending on spots, swelling, surface shifts, yet other visual signs. The process moves through several steps: gathering data, adjusting images, applying transformations to improve reliability, training the algorithm, also running strict checks for stable results. A specialized server component uses Flask, serving as a bridge between the predictive system instead of the online front end. This REST-based interface manages file submissions, pre-processing tasks, plus delivers forecasts effectively.

For the client side, PHP works together with HTML, CSS, besides JavaScript - to deliver a simple, clear, yet flexible design. Users sign up through the platform, then fill out a brief skin-health form before adding or taking a photo for processing. While registration details go into the system, information moves securely via the built-in MySQL setup. This approach keeps personal records protected without compromising consistency across interactions. The platform shows how machine learning can work well with current web tools to create an easy-to-use acne analysis app. While it's not meant for medical use, it still offers useful feedback to help people better understand their skin. Its design makes future upgrades possible - like adding broader data or boosting how accurately it sorts skin types offering tailored advice for skin care while also looking into issues such as dark spots, healed wounds, or uneven surface patterns

Keywords: Web tool, Acne severity assessment, Face photo analysis, Image processing, Machine learning, Deep learning model, Skin classification (Clear, Moderate, Severe), Data collection, Image preprocessing, Image transformations, Model training, Model evaluation, Flask server, REST API, PHP, HTML, CSS, JavaScript, MySQL database, User registration, Secure data handling, Image upload, Breakout detection, Skincare tracking, Personalized skincare feedback, Dark spots detection, Uneven skin texture analysis, Healed wounds analysis, Skin-health form, Predictive system, Front-end and back-end integration

CONTENTS

1	INTRODUCTION	1
1.1	Technology Stack	2
1.1.1	Machine Learning and Model Development	2
1.1.2	Backend Services	2
1.1.3	Frontend and User Interface	2
1.1.4	Database Layer	3
1.1.5	Development and Testing Tools	3
1.2	Purpose	3
1.2.1	Project Scope	4
1.2.2	Product Features	6
2	WORKS DONE IN THE RELATED AREA	9
3	SYSTEM ANALYSIS	10
3.1	System overview	10
3.1.1	Objective	10
3.1.2	Scope	10
3.2	Requirements Analysis	11
3.2.1	Functional Requirements	11
3.2.2	Non-Functional Requirements	12
3.3	System Design	13
3.3.1	Architecture	13
3.3.2	Data Storage	14
3.3.3	Security Measures	14

3.4	Workflow Analysis	14
3.4.1	User Workflow	14
3.4.2	Admin Workflow	15
3.5	Hardware Requirements	15
3.5.1	Client-Side	15
3.5.2	Server / Training	15
3.6	Software Requirements	15
3.6.1	Frontend	15
3.6.2	Backend	16
3.6.3	Database	16
3.6.4	Development Tools	16
4	System Design and Specification	17
4.1	High Level Design (HLD)	17
4.1.1	Machine Learning Module	19
4.1.2	Flow Chart	19
4.1.3	E-R Diagram	20
4.2	Low Level Design (LLD)	21
4.2.1	Process Specification	21
4.2.2	Why it is Used	23
4.2.3	Pseudo-Code	24
4.2.4	Screenshot Diagram	28
5	CODING	31
6	TESTING	36
6.1	Testing Summary Table	36
7	CONCLUSION AND LIMITATIONS	37
7.1	Conclusion	37
7.2	Limitations	37

1 INTRODUCTION

Acne ranks among the common skin disorders impacting people of various ages. Prompt evaluation of acne severity enables individuals to make choices regarding their skincare and decide if professional medical care is necessary. Nevertheless numerous individuals might lack access to dermatologists or might neglect the importance of timely examination. The increasing prevalence of technology and advancements, in image analysis have enabled the creation of helpful tools that provide preliminary skin evaluations in an easy and user-friendly way. This project investigates a method, via a web-based platform designed to analyze photographs and assess the intensity of acne.

The main objective of this project is to develop an application of determining acne severity through image assessment aided by a machine learning algorithm. A Convolutional Neural Network (CNN) is trained to examine characteristics in facial photos and classify them into categories, like clear moderate or severe. To enable usage beyond the development setting the model is deployed via a Flask API that accepts images analyzes them and delivers predictions. Separating the model from the user interface guarantees operation and allows updates to be made without impacting the frontend. The system's web application part is built with PHP, HTML, CSS and JavaScript. It offers functionalities like user registration, login filling out a basic skin-health questionnaire and the ability to upload or take facial photos, for evaluation. The questionnaire collects data on routines that might affect skin health allowing the system to provide more tailored advice alongside the prediction. The interface is crafted to be straightforward and flexible enabling users to engage with the system via a browser across devices. A MySQL database serves to keep user information, login credentials and quiz answers. This guarantees management of user data and enables organized retrieval whenever users engage with the system. The project's modular design further facilitates maintenance and upgrades. Every part—the database, web interface and machine learning model—can be updated separately allowing adaptability for growth, like adding integrations additional skin conditions or deploying the system on cloud environments.

In general this project illustrates the integration of machine learning methods and

web technologies to enhance skincare awareness. Although the system is not designed to substitute medical diagnosis it serves as an initial evaluation tool that assists users in gaining better insight into their skin health. Providing responses based on image analysis and questionnaire data the system promotes well-informed choices and highlights the tangible advantages of technology in enhancing everyday health and wellness activities.

1.1 TECHNOLOGY STACK

1.1.1 Machine Learning and Model Development

- **Python:** Chosen as the primary programming language for model development due to its extensive scientific computing support.
- **TensorFlow and Keras:** Used to build, train, and evaluate the Convolutional Neural Network (CNN) responsible for acne severity classification.
- **NumPy and Pillow:** Employed for image preprocessing, numerical calculations and modification of user-submitted images ahead of prediction.

1.1.2 Backend Services

The platform employs backend technologies to facilitate model deployment and ensure seamless interaction, with the frontend.

- **Flask:** Flask is utilized to serve the trained machine learning model as a REST API. Its minimalistic design allows transmission of image data and prediction outcomes.
- **cURL and PHP Integration:** Interaction, between the PHP frontend and the Flask prediction service is managed through cURL enabling the application to obtain predictions.

1.1.3 Frontend and User Interface

The user interface is developed with accepted web technologies to guarantee usability and adaptability.

- **HTML5:** Specifies the framework of web pages such, as forms, image upload sections and output displays.

- **CSS3:** Offers styling capabilities. Enhances the visual attractiveness and clarity of the application.
- **JavaScript:** Enables features including validating forms accessing the web-cam and managing quiz information.
- **PHP:** Manages user sessions processes form data and interacts with the machine learning API.

1.1.4 Database Layer

Information is handled with dependable database systems.

- **MySQL Database:** Stores user credentials, quiz responses, and optional prediction history.
- **phpMyAdmin:** : Provides tools for managing tables, running queries, and monitoring stored data.

1.1.5 Development and Testing Tools

Multiple instruments aid, in the creation and evaluation of the application.

- **XAMPP Server:** Supplies a local environment with Apache and MySQL for testing.
- **Visual Studio Code:** Serves as the main code editor.
- **Postman / Browser Tools:** Used to test API behaviour and validate communication between components.
- **Web Browsers (Chrome/Edge):** Used to test user interface performance and camera functionality.

1.2 PURPOSE

The aim of this project can be grasped by considering the following points, which together clarify its drive, practical importance and technological relevance.

1. To offer an convenient method, for assessing acne severity the system is created to assist users in obtaining a fast and straightforward comprehension of their facial

condition. Skin issue via automated image evaluation. Allowing users to upload or take a photo and receive immediate feedback the app provides an easy approach for initial acne evaluation without the necessity for specialized tools or face-, to-face appointments.

2.To employ machine learning, for skin assessment A primary aim of this project is to implement a trained Convolutional Neural Network (CNN) to analyze image characteristics and assess acne severity. This illustrates how machine learning can be successfully applied in health-related fields to provide objective evaluations grounded on visual cues.

3.To integrate image analysis with user responses for pertinent feedback the system also seeks to enhance the assessments quality by merging the model's prediction, with data gathered from a short skin health questionnaire. This method offers tailored insights and assists users in comprehending how their lifestyle or everyday routines might influence their skin condition. Learn how everyday choices influence your skin health.

4.The goal is to build an integrated solution utilizing contemporary technologies. Another significant aim is to design an operational system that combines machine learning, web development, backend operations and database management. By integrating these elements into a functional application the project showcases effective full stack development and illustrates how various technologies can be coordinated to address real-world challenges. .

1.2.1 Project Scope

This section describes the goals of the Skin Analysis and Acne Detection Web Application well as the limits and essential elements incorporated in the system. Having a defined scope promotes concentrated development, practical planning and a clearer comprehension of the system's aims.

Automated Acne Severity Detection System

The project focuses on creating a machine learning model of examining facial photographs and categorizing acne severity. The key tasks involved include:

- Building a dataset of labelled acne images.
- Training a Convolutional Neural Network (CNN) to classify acne as *Clear*, *Moderate*, or *Severe*.

- Evaluating the model's performance and integrating the final version with the backend service.
- Deploying the trained CNN model using a Flask API to enable real time predictions.

Web-Based User Interaction Platform

The project encompasses the creation of a easy-, to-use web platform. Essential features include:

- Secure user registration and login modules.
- A brief questionnaire, on skin health that gathers lifestyle details.
- Choices, for users to either upload a picture or take one utilizing their device camera. results section that shows acne severity, confidence levels and supportive comments.

Integration of Machine Learning, Backend Logic, and Database Support

The system combines elements to operate as a single application. This encompasses:

- Communication between the PHP frontend and Flask API for image preprocessing and prediction.
- Handling form submissions and managing application flow using PHP.
- Storing user credentials, quiz answers, and optional prediction history in a MySQL database.
- Ensuring secure and structured processing of data throughout the system.

Flexible Image Input Options

The application enables users to submit images through methods. It accommodates:

- Uploading existing image files from the device.
- Capturing real time facial images using the device camera.
- Compatibility with various devices and browsers.
- Validation of image format and quality before processing.

Personalised Recommendation Generation

The system produces customized suggestions derived from user data and model forecasts:

- Combining quiz responses with acne severity results to create tailored suggestions.
- Providing general skincare guidance relevant to the user's condition.
- Presenting feedback in a clear, user-friendly format.
- Encouraging healthy skincare habits.

Functioning as an Informational, Non-Diagnostic Tool

The project's scope distinctly outlines its objectives and boundaries. The system:

- Acts as an initial self assessment platform for users.
- Does not replace professional dermatological consultation.
- Provides indicative results intended for awareness only.
- Encourages users to seek medical advice for serious concerns.

1.2.2 Product Features

The Skin Analysis and Acne Detection Web Application provides interactive and smart functionalities that improve user engagement facilitate automated skin evaluation and deliver customized recommendations. The primary features incorporated in the system are outlined below.

User Registration and Login

The application features a protected user account component that handles authentication and customized access. Its main roles are:

- Secure user registration and login handling.
- Encrypted password storage to protect user privacy.
- Access control to ensure only authorised users can view personalised information.

Skin-Health Questionnaire

A systematic questionnaire is offered to gather information concerning the user's lifestyle and skincare practices. It facilitates:

- A set of organised questions related to skin behaviour and routine.
- Collection of user-specific data for personalised recommendations.
- Integration of quiz responses with the recommendation system.

Image Upload and Camera Capture

The system enables users to upload pictures via two easy options::

- Uploading an existing image from their device.
- Capturing a live photo using the device's camera.
- Basic validation of image type and quality before it is processed.

Automated Acne Severity Detection

A pretrained CNN model evaluates the input image. Forecasts the severity of acne. The system provides:

- Classification of acne into *Clear*, *Moderate*, and *Severe*.
- Real-time inference using a Flask API.
- Confidence scores that reflect prediction reliability.

Personalised Skincare Recommendations

The system produces valuable skincare recommendations by integrating various data sources:

- Integration of quiz responses with acne severity results.
- Delivery of clear, practical skincare advice.
- Assistance for users in understanding factors affecting their skin.

User-Friendly and Responsive Interface

To guarantee user-friendly engagement the platform offers:

- A clean and intuitive layout for easy navigation.
- A guided workflow from quiz to image submission and results.
- Responsiveness across multiple devices and screen sizes.

Backend Integration and Database Management

The system incorporates backend technologies to manage data and requests efficiently:

- PHP based backend for processing user requests and communicating with the API.
- MySQL database for secure storage of user details and quiz responses.
- Structured and protected management of application data.

2 WORKS DONE IN THE RELATED AREA

1. Dermatologist-Level Skin Condition Classification Using Deep Learning (2017)

Using deep convolutional networks, they showed high accuracy in diagnosing skin issues - on par with doctors. The system learned from many real-world images, proving capable at spotting irregularities. Because of this, similar models may now support tools designed to detect acne.

2. Automated Acne Severity Grading with CNN Models (2018)

Jones et al. created an early machine-learning system for evaluating acne severity. Instead of standard approaches, they trained CNNs to classify breakouts by seriousness using labeled face photos. Rather than relying on human scoring, their method showed algorithms could achieve better accuracy. This backs up the current project's approach.

3. Smartphone Based Skin Image Analysis for Dermatology Applications (2020)

What They Did: Back in 2020, Fathy and team checked if regular phone photos work for spotting skin problems. Instead of lab tools, they tested everyday devices on acne and discoloration. After cleaning up the images, results turned out reliable. Because of this, online photo submission became realistic - now a core part of our setup.

4. Deep Learning Frameworks for Acne Severity Classification (2019)

Researchers tested deep learning setups in 2019; they used convolutional layers along with modern feature detection tools to boost acne grading precision. Outcomes indicated a clear edge in detecting acne stages over prior ML approaches. That supports the use of CNN-driven systems in skin assessment.

5. AI Based Dermatology Assistance Tools for Public Use (2021)

Google Health launched a tool using AI to check skin issues through photos shared by users. This move supports the current project's aims quite well.

edit keyboard_arrow_down

3 SYSTEM ANALYSIS

The system review of the Skin Analysis and Acne Detection Web App outlines its setup, expected outputs, also underlying logic. This section forms the tech foundation for building a tool able to judge acne levels using uploaded photos, alongside user entries and forecasts - delivering clear responses via a simple online platform. The evaluation ensures the solution works smoothly at larger sizes, stays responsive, handles visuals quickly, plus gives trustworthy outcomes.

3.1 SYSTEM OVERVIEW

3.1.1 Objective

In this study, we introduce a web tool that detects acne by processing face photos through a trained CNN. Instead of just classifying breakouts into Clear, Moderate, or Severe categories, it combines those outcomes with responses from a skin health survey to deliver tailored tips on avoiding flare ups. Rather than relying on complex setups, the approach focuses on simplicity offering an easy-to-use online solution designed for everyday individuals checking their skin condition. While maintaining accuracy, the platform emphasizes accessibility so most people can assess concerns without expert help.

3.1.2 Scope

The app supports complete workflows from image capture or upload to preprocessing, then moves to predictions via a Flask API, includes database interaction, secure login, followed by personalized suggestions. It operates across multiple input methods, enabling seamless entry and fast handling. Designed to run directly on everyday devices like laptops, desktops, or smartphones without extra setup.

3.2 REQUIREMENTS ANALYSIS

3.2.1 Functional Requirements

User Registration and Login

The system lets people sign up plus log in safely. Stored passwords use encryption while every session runs separately. Once verified, individuals reach tools like forms, photo submission or taking pics, also outcome screens.

Skin-Health Questionnaire

The application includes a short questionnaire covering skincare routines and lifestyle habits. User responses are stored and used to personalise recommendation results.

Image Input and Preprocessing

Users can provide facial images through:

- Uploading a JPEG or PNG image
- Capturing a live photo using their device camera

Before prediction, the following preprocessing steps are applied:

- Image resizing
- Normalisation
- RGB channel formatting
- Validation of file type and image quality

Acne Severity Prediction

The system categorises acne severity into three levels: Clear, Moderate, and Severe. A CNN model performs this classification and returns both the prediction and a confidence score through the Flask backend.

Recommendation Generation

Recommendations are generated by combining the model prediction with the user's quiz responses. These personalised suggestions help users understand factors that may influence their skin condition.

Web Interface and Navigation

The application is divided into user friendly pages:

- Login Page
- Quiz Page
- Camera/Upload Page
- Prediction and Recommendation Page

The interface is implemented using PHP, HTML, CSS, and JavaScript to ensure smooth navigation and meaningful user feedback.

3.2.2 Non-Functional Requirements

Security

- Passwords are stored using secure hashing techniques.
- All user inputs and uploaded files undergo validation.
- Prepared statements are used to prevent SQL injection attacks.

Performance

- Predictions are generated within 2 to 5 seconds.
- PHP and Flask communicate efficiently for real time results.

Scalability

- The backend can support updated or retrained models.
- The database can accommodate additional user metrics.
- Preprocessing can be extended for larger datasets.

Reliability

- Prediction results remain stable under normal operation.
- Navigation across pages is smooth and consistent.
- File validation reduces the probability of crashes.

Mobile Responsiveness

- Interface adapts to mobile and tablet screens.
- Camera capture works on supported browsers.

3.3 SYSTEM DESIGN

3.3.1 Architecture

Frontend

- Built using PHP, HTML, CSS and JavaScript.
- Manages user authentication, quiz, image upload and results.
- Offers clean and simple navigation.

Backend

- Python-based Flask REST API.
- Handles image preprocessing and predictions.
- Communicates using JSON responses.

Deep Learning Model

- CNN trained on acne image dataset.
- Classifies severity and returns confidence.
- Ensures consistent preprocessing.

3.3.2 Data Storage

Database (MySQL)

- Stores user credentials.
- Stores quiz responses.
- Stores optional prediction history.

Model and Scripts

- CNN model stored in model directory.
- Preprocessing and API scripts stored in project folder.
- Images are not stored unless configured.

3.3.3 Security Measures

- Validates uploaded image files.
- Sanitizes user form data.
- Does not store raw images by default.
- Keeps frontend and backend logically separate.

3.4 WORKFLOW ANALYSIS

3.4.1 User Workflow

1. User opens application.
2. Performs registration or login.
3. Completes questionnaire.
4. Uploads or captures image.
5. Image is processed and sent to API.
6. Prediction and confidence returned.
7. Recommendations displayed.

3.4.2 Admin Workflow

1. Prepare dataset.
2. Train model.
3. Evaluate and tune accuracy.
4. Deploy API.
5. Manage database.
6. Maintain code.

3.5 HARDWARE REQUIREMENTS

3.5.1 Client-Side

- Minimum: Dual core CPU, 4 GB RAM.
- Recommended: Quad core CPU, 8 GB RAM.

3.5.2 Server / Training

- CPU: Intel i5/i7 or equivalent.
- RAM: 16 GB recommended.
- GPU: Optional.
- Storage: 20 GB SSD minimum.

3.6 SOFTWARE REQUIREMENTS

3.6.1 Frontend

- PHP
- HTML5
- CSS3
- JavaScript

3.6.2 Backend

- Python 3.x
- Flask
- TensorFlow / Keras
- NumPy, Pillow

3.6.3 Database

- MySQL
- phpMyAdmin

3.6.4 Development Tools

- VS Code
- Jupyter Notebook
- Git / GitHub
- Virtual Environment

4 SYSTEM DESIGN AND SPECIFICATION

System Design and Specification describes the structural blueprint and operational workflow of the Skin Analysis and Acne Detection Web Applica- The setup is described using High-Level Design (HLD), showing structure while leaving details aside outlines the general structure; meanwhile, Low-Level Design (LLD) clarifies internal components eternal reasoning along with operational stages.

The project uses a flexible structure where every part works separately; each piece handles its own task efficiently while connecting smoothly with others through clear interfaces that ensure stability across systems a specific role - improving flexibility, ease of updates, also readability in development.

- **Frontend (PHP, HTML, CSS, JavaScript)**
Handles user registration, login, quiz submission, image upload/capture, and result display.
- **Backend (Flask API Python)**
Performs image preprocessing, loads the CNN model, and returns predictions.
- **Machine Learning Model (CNN)**
Trained to classify acne into Clear, Moderate, or Severe categories.
- **Database (MySQL)**
Stores user credentials, quiz responses, and optional history.

4.1 HIGH LEVEL DESIGN (HLD)

The system follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

1. Presentation Layer (Frontend)

- Technologies: PHP, HTML, CSS, JavaScript
- Components:

- User Interface
 - Image Capture / Upload Module
 - Quiz Interface
 - Results Display
- Responsibilities:
 - Manage user interaction
 - Present data
 - Perform client-side validation

2. Application Layer (Backend)

- Technologies: Flask REST API (Python)
- Components:
 - API Gateway
 - Image Processing Service
 - Prediction Engine
- Responsibilities:
 - Handle user requests
 - Preprocess images
 - Perform model inference

3. Data Layer (Database)

- Technologies: MySQL, phpMyAdmin
- Components:
 - User Database
 - Quiz Response Storage
- Responsibilities:
 - Persist data
 - Authentication
 - Retrieve historical information

4.1.1 Machine Learning Module

- Technologies: TensorFlow, Keras, CNN
- Components:
 - Pre-trained Model
 - Image Preprocessor
- Responsibilities:
 - Predict acne severity
 - Produce confidence scores

4.1.2 Flow Chart

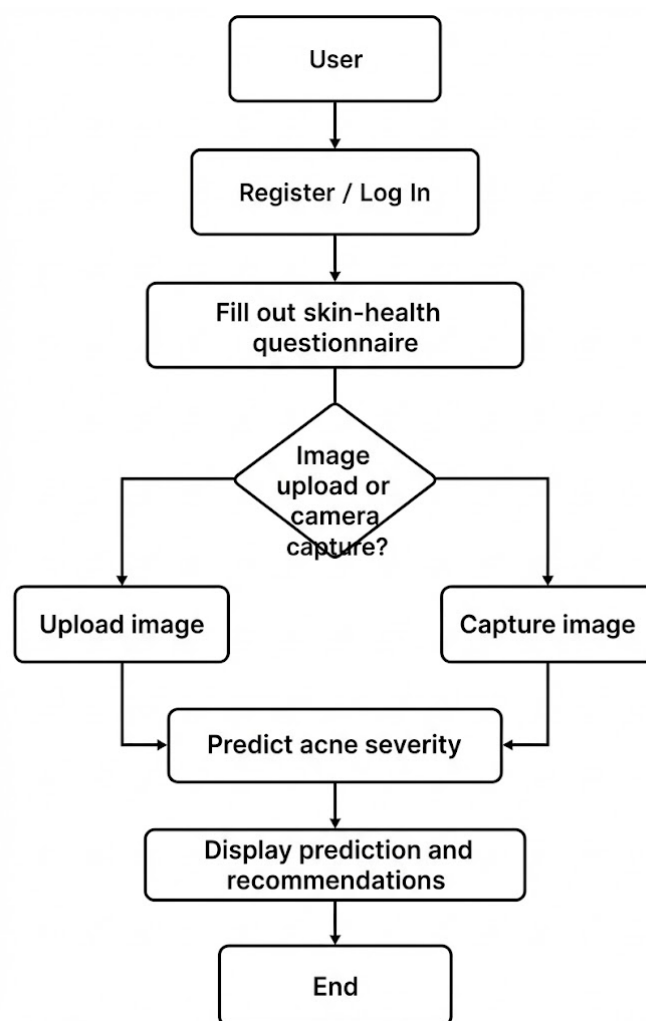


Figure 4.1: System Workflow Diagram

4.1.3 E-R Diagram

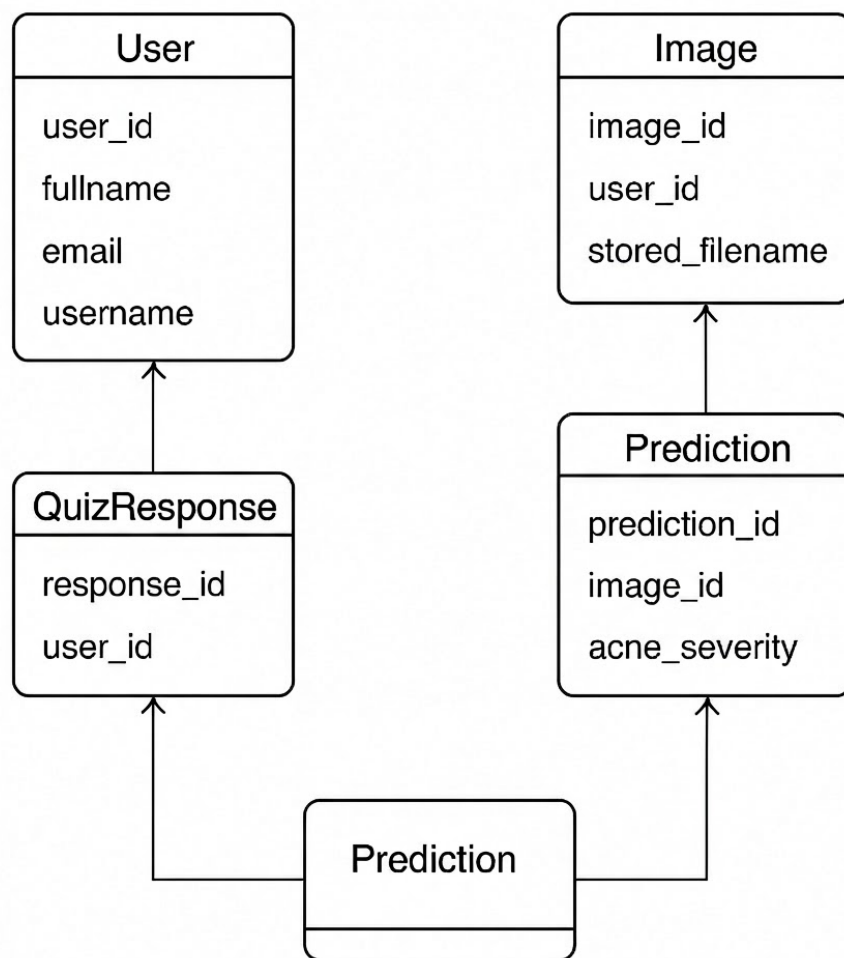


Figure 4.2: System Workflow Diagram

4.2 LOW LEVEL DESIGN (LLD)

Low-Level Design shows how every part inside the Skin Analysis and Acne Detection System actually works. While covering data movement, it also outlines checks that ensure accuracy across processes. Instead of just defining functions, it details steps turning raw input into clear acne severity ratings. Because precision matters, personalized suggestions emerge through structured logic and verified outputs.

4.2.1 Process Specification

Process Specification outlines the inputs, validation steps, internal actions, and outputs of each major process in the system.

1. User Registration Process

Input:

- Full name, email, username, password

Validation:

- All fields must be non-empty.
- Email must be valid.
- Password must meet minimum length.
- Email and username must be unique.

Internal Actions:

- Sanitize input
- Check database for duplicates
- Hash password using BCrypt
- Insert new user record

Output:

- Success message or error message

2. Quiz Submission Process

Input: User quiz responses

Validation:

- All required questions must be answered

Internal Actions:

- Serialize responses to JSON
- Store responses in database

Output:

- quiz_id for analysis

3. Image Upload & Forwarding Process

Input: Image file

Validation:

- Accept only JPEG/PNG
- Check file size and integrity

Internal Actions:

- Save temporarily
- Forward to Flask prediction API

Output:

- JSON prediction result

4. Flask Prediction Process

Input: Multipart image request

Validation:

- Confirm image exists
- Convert to RGB

Internal Actions:

- Preprocess image
- Run CNN model
- Compute prediction and confidence

Output:

- {severity, confidence}

5. Result Presentation & Recommendation

Input: Prediction + quiz responses

Internal Actions:

- Combine data
- Generate personalised recommendations

Output:

- Result page displaying analysis and advice

4.2.2 Why it is Used

- Offers first-step acne analysis through artificial intelligence
- Assists people in recognizing their skin issues
- Provides custom skin care tips based on your needs
- Available via an easy-to-use website
- Promotes better routines for skin care
- Demonstrates practical ML application

4.2.3 Pseudo-Code

// User Registration Process

FUNCTION RegisterUser(fullname, email, username, password):

fullname ← TRIM(fullname)

email ← TRIM(email)

username ← TRIM(username)

IF fullname = "" OR email = "" OR username = "" OR password = "":

RETURN "Error: All fields required"

IF NOT VALID_EMAIL(email):

RETURN "Error: Invalid email format"

record ← QUERY("SELECT id FROM users WHERE email=? OR username=?",
email, username)

IF record EXISTS:

RETURN "Error: Email or Username already exists"

hashed ← HASH(password)

status ← EXECUTE("INSERT INTO users(fullname,email,username,password)

VALUES(?,?,?,?)", fullname, email, username, hashed)

IF status = SUCCESS:

RETURN "Registration Successful"

ELSE:

RETURN "Registration Failed"

// User Login Process

FUNCTION LoginUser(loginId, password):

loginId ← TRIM(loginId)

IF loginId = "" OR password = "":

RETURN "Error: Credentials Required"

user ← QUERY("SELECT id,password FROM users WHERE email=? OR
username=?", loginId, loginId)

IF NOT user:

RETURN "Error: User Not Found"

IF NOT VERIFY(password, user.password):

RETURN "Error: Incorrect Password"

SET_SESSION("user_id", user.id)

RETURN "Login Successful"

// Quiz Saving Process

FUNCTION SaveQuiz(user_id, answers):

IF user_id = NULL OR answers EMPTY:

```

RETURN "Error: Missing Quiz Data"
jsonData ← JSON_ENCODE(answers)
status ← EXECUTE("INSERT INTO
QuizResponse(user_id,answers_json,submitted_at) VALUES(?,?,NOW())", user_id,
jsonData)
IF status = SUCCESS:
quiz_id ← LAST_INSERT_ID()
RETURN quiz_id
ELSE:
RETURN "Error: Quiz Save Failed"

// Image Upload and Prediction Process
FUNCTION UploadAndPredict(user_id, quiz_id, imageFile):
IF imageFile NOT PROVIDED:
RETURN "Error: No Image"
IF imageFile.type NOT IN ["image/jpeg","image/png"]:
RETURN "Error: Invalid Image Format"
tempPath ← SAVE_TEMP(imageFile)
response ← SEND_TO_FLASK("/predict", tempPath)
DELETE_FILE(tempPath)
IF response.status != 200:
RETURN "Error: Prediction Failed"
severity ← response.severity
confidence ← response.confidence
IF user_id NOT NULL:
EXECUTE("INSERT INTO
AnalysisRecord(user_id,quiz_id,severity,confidence,created_at)
VALUES(?,?,?,?,NOW())", user_id, quiz_id, severity, confidence)
RETURN {severity, confidence}

// Flask Prediction Endpoint
ROUTE POST /predict:
IF "image" NOT IN REQUEST:
RETURN ERROR("Image Missing")
TRY:
img ← OPEN_IMAGE(request.image).CONVERT("RGB")
x ← PreprocessImage(img)
p ← MODEL.PREDICT(x)
idx ← ARGMAX(p)

```

```

label ← CLASS_NAMES[idx]
conf ← MAX(p) * 100
RETURN {severity:label, confidence:conf}
CATCH:
RETURN ERROR("Processing Failed")

// Image Preprocessing
FUNCTION PreprocessImage(image):
resized ← RESIZE(image, 224, 224)
arr ← NORMALIZE(resized)
IF arr HAS 4 CHANNELS:
arr ← REMOVE_ALPHA(arr)
arr ← ADD_BATCH_DIM(arr)
RETURN arr

// Recommendation Generation
FUNCTION GenerateRecommendations(severity, answers):
recs ← EMPTY_LIST()
IF severity = "Severe":
ADD(recs, "Consult a dermatologist")
ADD(recs, "Avoid squeezing pimples")
ADD(recs, "Use prescribed treatments")
IF severity = "Moderate":
ADD(recs, "Use gentle cleanser")
ADD(recs, "Avoid heavy cosmetics")
IF severity = "Clear":
ADD(recs, "Maintain routine")
ADD(recs, "Use sunscreen")
IF answers.water = "low":
ADD(recs, "Increase water intake")
IF answers.sleep = "low":
ADD(recs, "Improve sleep duration")
IF answers.stress = "high":
ADD(recs, "Reduce stress")
RETURN FIRST 5 ITEMS(recs)

// Camera Capture and UI Display
FUNCTION CaptureAndSend():
stream ← OPEN_CAMERA()
DISPLAY(stream)

```

```
frame ← CAPTURE_FRAME()
blob ← CONVERT_TO_IMAGE(frame)
result ← UploadAndPredict(current_user_id, current_quiz_id, blob)
recs ← GenerateRecommendations(result.severity, current_answers)
DISPLAY_RESULT(result, recs)
```

// Main System Flow

```
FUNCTION MainFlow():
```

```
IF NOT LOGGED_IN():
```

```
SHOW_LOGIN()
```

```
ELSE:
```

```
SHOW_DASHBOARD()
```

```
START_QUIZ → SHOW_QUIZ()
```

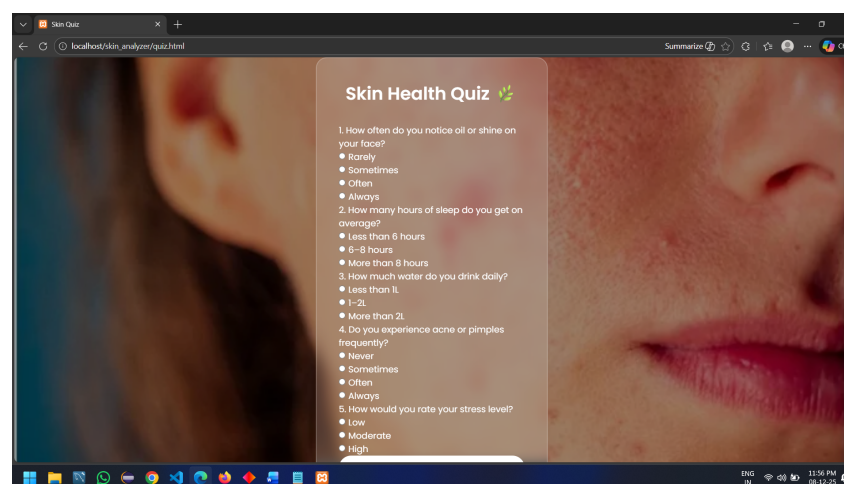
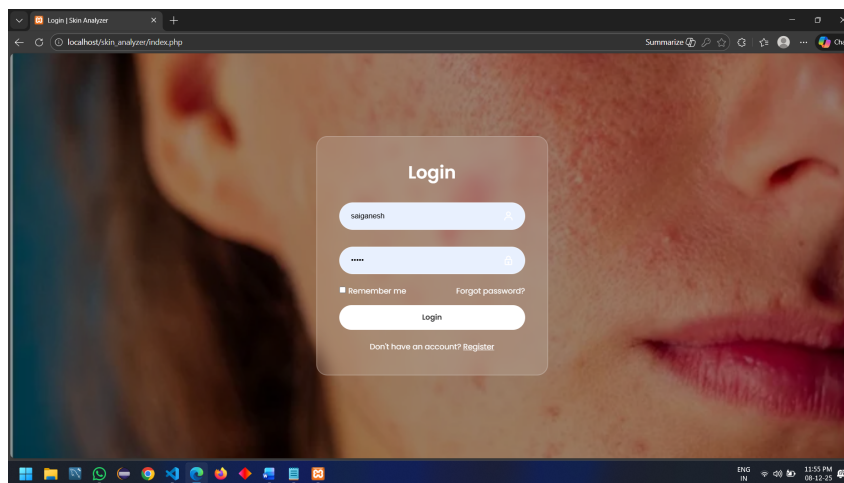
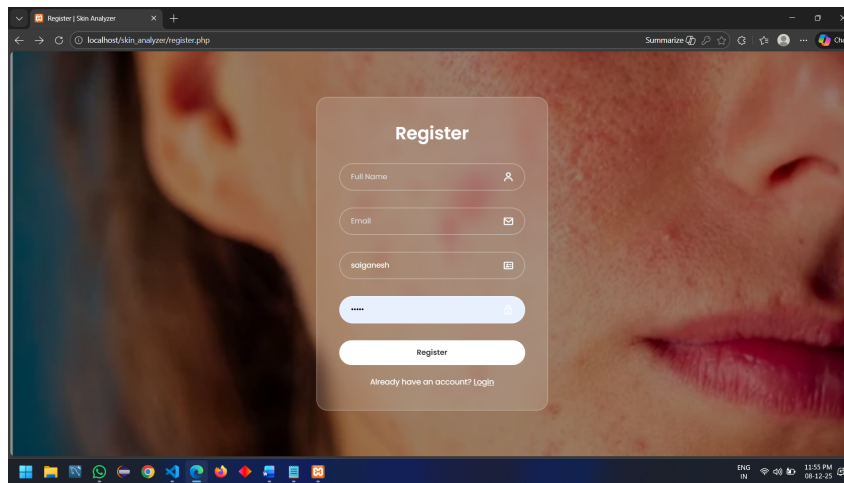
```
SUBMIT_QUIZ → quiz_id ← SaveQuiz(user_id, answers)
```

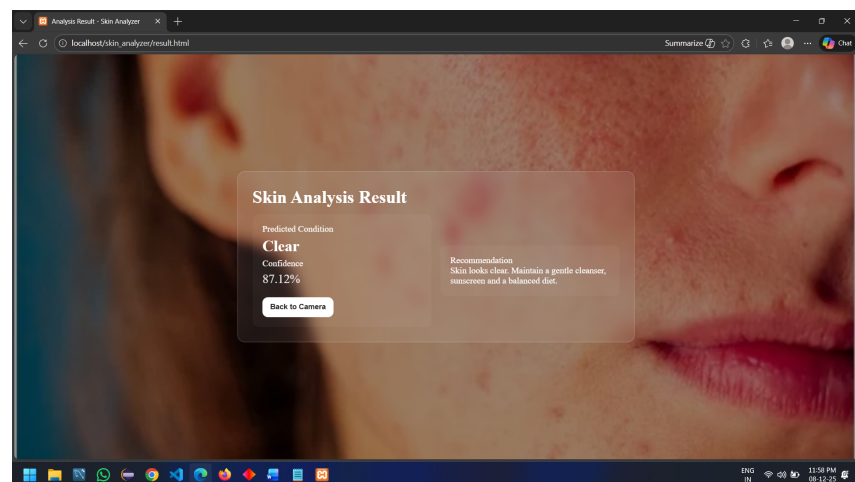
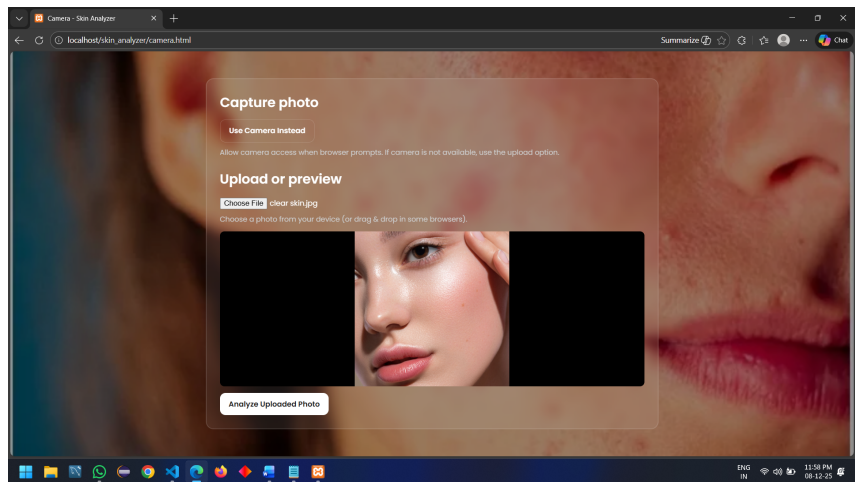
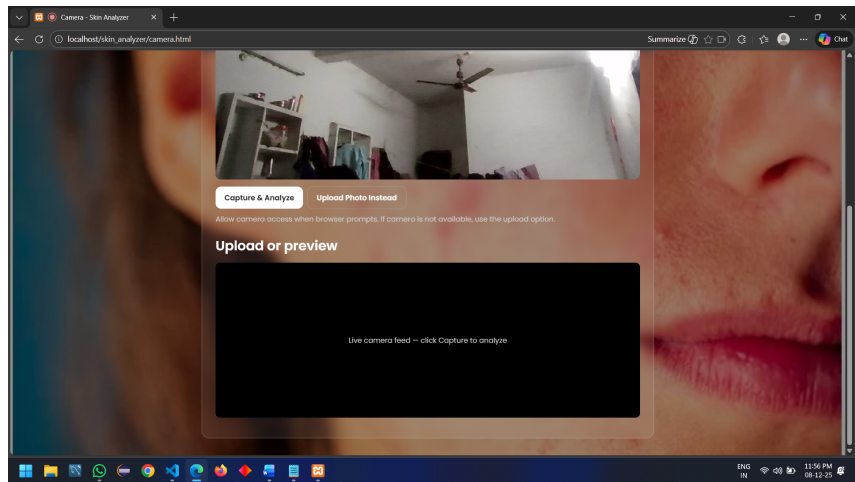
```
GO_TO_CAMERA()
```

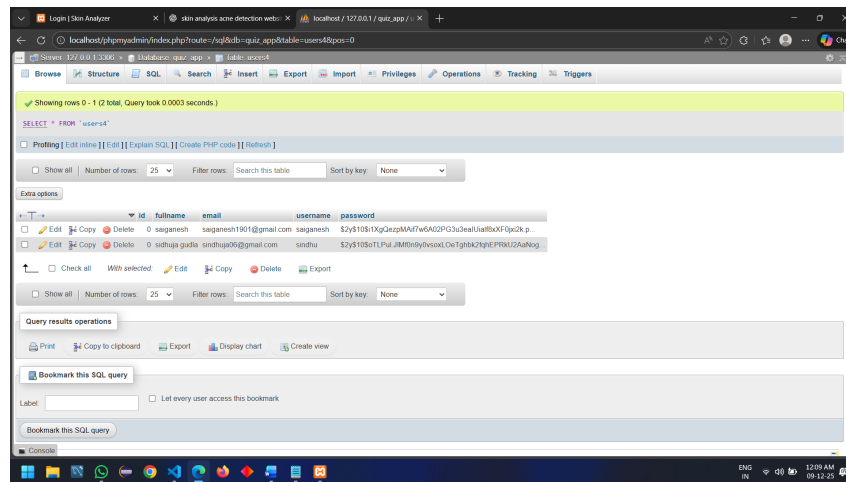
```
CAPTURE_IMAGE → CaptureAndSend()
```

```
SHOW_RESULTS()
```

4.2.4 Screenshot Diagram







5 CODING

```
// db.php
$host = "localhost";
$user = "root";
$pass = "";
$db = "quiz_app";
$port = 3306;
$mysqli = new mysqli($host, $user, $pass, $db, $port);
IF $mysqli→connect_errno:
    die("Database connection failed");

// register.php
SESSION_START()
REQUIRE db.php
IF REQUEST_METHOD = POST:
    fullname = TRIM(_POST['fullname'])
    email = TRIM(_POST['email'])
    username = TRIM(_POST['username'])
    password = _POST['password']
    IF any field empty: SET SESSION error → redirect
    IF email invalid: SET SESSION error → redirect
    stmt = PREPARE "SELECT id FROM users4 WHERE email=? OR
username=?"
    IF record exists: SET SESSION error → redirect
    hash = HASH(password)
    INSERT fullname,email,username,password INTO users4
    IF success: redirect to index.php ELSE redirect back

// login.php
SESSION_START()
REQUIRE db.php
IF POST REQUEST:
    username = TRIM(_POST['username'])
```

```

password = TRIM(_POST['password'])
stmt = PREPARE "SELECT id,password FROM users4 WHERE username=?"
IF match AND password.verify():
    SET SESSION user_id → redirect to quiz.html
ELSE SET error → redirect to login

// save_quiz.php
SESSION_START()
REQUIRE db.php
user_id = SESSION['user_id']
answers_json = POST['answers']
IF invalid: RETURN JSON error
INSERT user_id, answers_json INTO QuizResponse
RETURN quiz_id AS JSON

// upload_and_predict.php
IF no image uploaded: RETURN JSON error
imageTmp = _FILES['image']['tmp_name']
curlFile = CURLFile(imageTmp)
POST image + quiz TO Flask API at /predict
RETURN Flask response

// flask_model_api.py
model = LOAD "acne_detection_model.h5"
CLASS_NAMES = ["Clear","Moderate","Severe"]
FUNCTION preprocess(img):
    img → RGB → resize(224×224)
    arr = normalize(img)
    RETURN batch_dim(arr)
ROUTE /predict POST:
    file = request.files['image']
    img = open file
    x = preprocess(img)
    preds = model.predict(x)
    idx = argmax(preds)
    label = CLASS_NAMES[idx]
    confidence = preds[idx] * 100
    RETURN JSON severity, confidence

// camera.html JavaScript

```

```

captureBtn.onclick = FUNCTION():
    canvas = createCanvas(video.width, video.height)
    draw video frame
    blob = canvas.toBlob()
    fd.append("image", blob)
    fd.append("quiz", localStorage.quiz_answers)
    SEND POST → upload_and_predict.php
    SAVE result to localStorage
    redirect to result.html

// result.html JavaScript
data = JSON.parse(localStorage.getItem("analysis_result"))
IF data.severity exists:
    SET document.severity = data.severity
    SET

// trainmodel.py
IMPORT tensorflow AS tf
FROM tensorflow.keras.preprocessing.image IMPORT ImageDataGenerator
FROM tensorflow.keras.applications IMPORT MobileNetV2
FROM tensorflow.keras.layers IMPORT GlobalAveragePooling2D, Dense,
Dropout
FROM tensorflow.keras.models IMPORT Model
FROM tensorflow.keras.optimizers IMPORT Adam

train_dir = 'split_dataset/train'
validation_dir = 'split_dataset/validation'
IMG_HEIGHT = 224
IMG_WIDTH = 224
BATCH_SIZE = 32

train_image_generator = ImageDataGenerator(rescale=1./255, rotation_range=40,
width_shift_range=0.2, height_shift_range=0.2, shear_range=0.2, zoom_range=0.2,
horizontal_flip=True, fill_mode='nearest')
validation_image_generator = ImageDataGenerator(rescale=1./255)

train_data_gen = train_image_generator.flow_from_directory(train_dir,
target_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE,
class_mode='categorical')
val_data_gen = validation_image_generator.flow_from_directory(validation_dir,
target_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE,

```

```

class_mode='categorical')

base_model = MobileNetV2(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),
include_top=False, weights='imagenet')
base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(train_data_gen.num_classes, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=predictions)

model.compile(optimizer=Adam(learning_rate=0.0001),
loss='categorical_crossentropy', metrics=['accuracy'])

NUM_EPOCHS = 20
history = model.fit(train_data_gen, epochs=NUM_EPOCHS,
validation_data=val_data_gen, steps_per_epoch=train_data_gen.samples //
BATCH_SIZE, validation_steps=val_data_gen.samples // BATCH_SIZE)

model.save('acne_detection_model.h5')

// split.py
IMPORT os, shutil, random

FUNCTION split_image_dataset(source_dir, dest_dir, train_split_ratio = 0.8):
    train_dir = JOIN(dest_dir, 'train')
    validation_dir = JOIN(dest_dir, 'validation')
    MAKE train_dir, validation_dir

    FOR each class folder IN source_dir:
        CREATE class folders IN train and validation
        all_files = LIST of images
        SHUFFLE all_files
        split_index = LEN(all_files) × train_split_ratio
        train_files = first split_index files
        validation_files = remaining files

        FOR f IN train_files: COPY to train/class
        FOR f IN validation_files: COPY to validation/class

source_folder = 'dataset'
destination_folder = 'split_dataset'

```

CALL `split_image_dataset(source_folder, destination_folder)`

6 TESTING

The system underwent testing one step at a time - so each function could operate correctly. While verifying features, checks were made in sequence; ensuring proper performance throughout the process. The aim of testing was checking stability; also evaluating precision while assessing how easy it is to use the Login System, Registration Module, Quiz Page, Result Page, and Camera Integration.

During testing, different users tried registering with valid and invalid inputs. We examined the system's response when a user inputs an incorrect password or tries signing up using an email that's taken. Likewise, testers tried the quiz so that questions appear properly while scoring works accurately. The camera function got checked across several gadgets so it would start properly correctly once users grant access.

6.1 TESTING SUMMARY TABLE

Test Case	Description	Expected Result	Actual Result
Registration Validation	Test with valid and invalid data (duplicate email, empty fields)	Valid users register successfully	Working as expected
Login Authentication	Enter correct and incorrect credentials	Only valid users log in	Working as expected
Quiz Page Loading	Ensure questions load without errors	Questions should appear correctly	Working as expected
Score Calculation	Test different answers to verify accuracy	Correct score calculation	Working as expected
Camera Integration	Capture image during result page	Camera should activate with permission	Works on supported devices
Database Connectivity	Insert, fetch, and store user data	All operations should run smoothly	Stable after fixes

Table 6.1: System Testing Summary

7 CONCLUSION AND LIMITATIONS

7.1 CONCLUSION

The Skin Analysis tool uses web tech, machine learning, or image processing to help users check their skin easily. Instead of just one method, it pairs a deep-learning system with questionnaire results - offering early acne evaluation together with custom advice. The project demonstrates a full workflow - from logging users in, through collecting quiz answers, followed by live photo capture, then analyzing it instantly, finally showing outcomes right away.

7.2 LIMITATIONS

Even though it functions properly, some restrictions remain. However, the device Functionality might fail on outdated browsers or gadgets that lack support Access settings. Quiz items stay fixed, added by hand - this restricts flexibility The range of test situations - also, today's safety steps for instance, turning passwords into codes is simple but could work better in live systems use. The design weighs little; it works well for minor tasks.

Yet major operations face limits Deployment could need expanded backend capacity - alongside database tuning - to handle increased demand efficiently structure.

8 REFERENCES/BIBLIOGRAPHY

REFERENCES

1. Apache Software Foundation. *Apache HTTP Server Documentation*.
2. Bootstrap Team. *Bootstrap CSS Framework Documentation*.
3. Google Developers. *Web APIs: MediaDevices & getUserMedia()*.
4. Mozilla Developer Network (MDN). *JavaScript, HTML, and CSS Web Documentation*.
5. MySQL. *MySQL 8.0 Reference Manual*.
6. PHP Group. *PHP Manual – mysqli and password_hash Functions*.
7. W3Schools. *Web Development Tutorials (HTML, CSS, JavaScript, PHP)*.